

Verilog 模擬流程筆記

步驟 1–3：編譯

```
iverilog -o sim_out counter.v tb_counter.v
```

拆開看：

部分	意思
<code>iverilog</code>	編譯器
<code>-o sim_out</code>	output，把結果存成 <code>sim_out</code> 這個檔
<code>counter.v tb_counter.v</code>	要編譯的原始碼，設計檔和測試檔都要列進去

確認編譯結果

```
echo %ERRORLEVEL%
```

- 印出 **0** → 成功，往下走
- 印出**任何其他數字** → 失敗了，別往下走

[!!IMPORTANT] 這一步就是你上次踩到的坑：畫面一片安靜，但其實是失敗的。
`%ERRORLEVEL%` 是「上一個指令的結束碼」，在 Windows 上，**沒訊息不代表成功**，**0 才代表成功**。

養成習慣：每次覺得「怎麼好像沒反應」，就打這一行問問看。

步驟 4：跑模擬

```
vvp sim_out
```

`vvp` 是模擬器，`sim_out` 就是步驟 2 產生的那個檔。你會看到：

```
VCD info: dumpfile wave.vcd opened for output.  
時間=0   | 重置=1 | 計數值=xxxx ( x)  
時間=5   | 重置=1 | 計數值=0000 ( 0)  
時間=15  | 重置=0 | 計數值=0001 ( 1)  
...  
時間=105 | 重置=0 | 計數值=1010 (10)
```

[!NOTE] 第一行 `dumpfile wave.vcd opened` 很重要 —— 代表波形檔開始寫了。

步驟 5：確認波形檔生出來了

```
dir wave.vcd
```

有看到 `wave.vcd` 和它的大小就對了。找不到的話，是 testbench 裡缺 `$dumpfile` / `$dumpvars`。

步驟 6：看波形

```
gtkwave wave.vcd
```

GTKWave 視窗會跳出來。進去之後：

1. 左上角 SST 區塊點一下 `tb_counter`
2. 下方會列出訊號：`clk`、`reset`、`count`
3. 選起來，按 **Insert** 按鈕（或直接拖到右邊）
4. 用 **Zoom Fit**（工具列那個放大鏡）把整段波形縮到看得見

每次模擬的完整流程

之後每次要模擬，就是這四行：

```
iverilog -o sim_out counter.v tb_counter.v  
echo %ERRORLEVEL%  
vvp sim_out  
gtkwave wave.vcd
```

[!WARNING] 改完 `.v` 檔之後一定要從第一行重跑，不能只跑 `vvp` ——
`sim_out` 還是舊的，你會看到改之前的結果，這是初學最常見的困惑。

[!TIP] cmd 裡按 `↑` 上方向鍵可以叫回上一個打過的指令，不用每次重打。